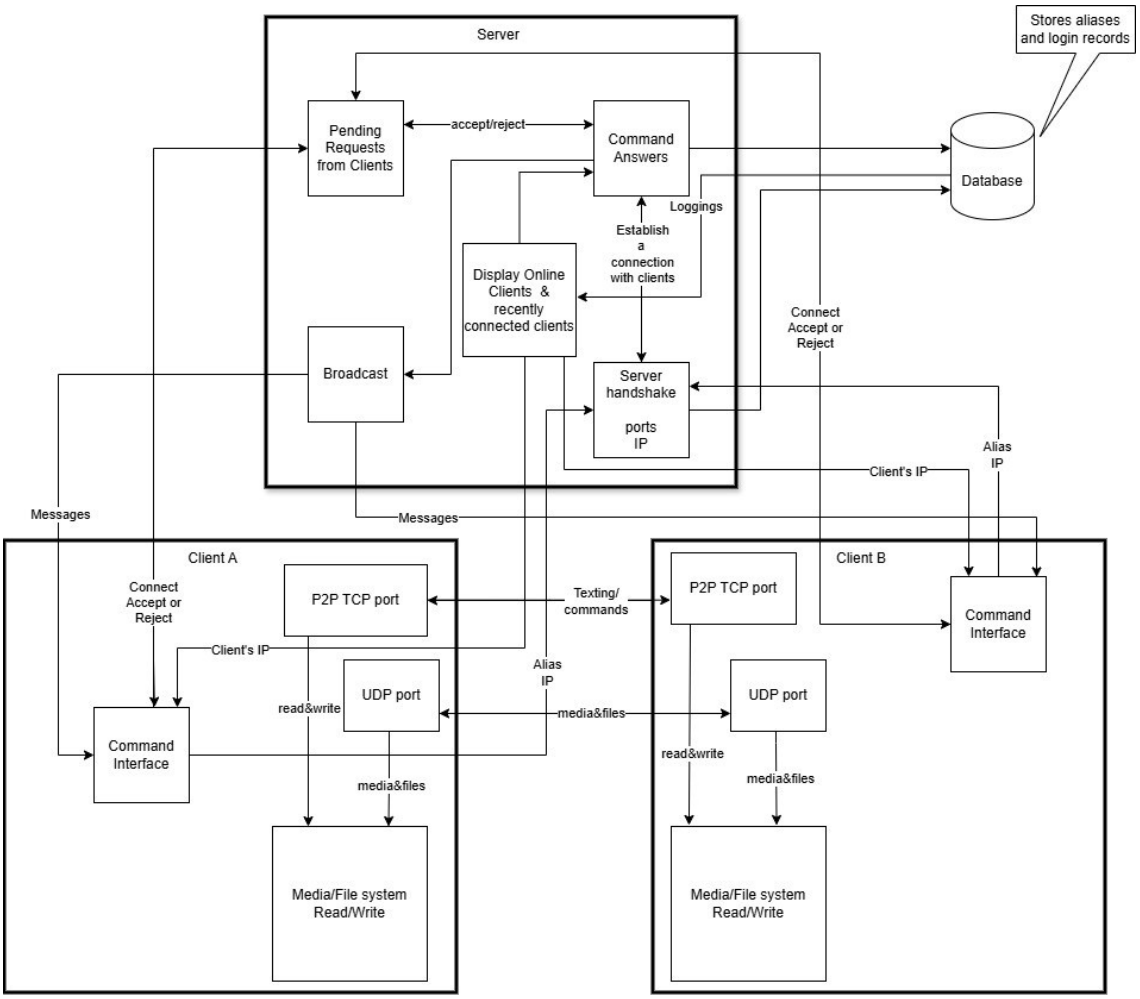
Stage 1: Design and Protocol Specification

Deliverables

1. System architecture diagram



2. Application-level protocol specifications

Client–Server Communication

Under client–server communication, the server manages user registration, discovery commands, and connection coordination.

1. User Registration

The server is responsible for registering users when they first connect. During registration, the server collects the client's username, IP address, and port information, then stores this data in the server database. This allows the server to uniquely identify each client and maintain a list of active users.

2. Discovery Commands

The server provides discovery commands that clients can use to obtain information about other users. For example, a client may request:

- A list of currently online users
- The IP address and port of another client
- Availability status of a specific user
- The general commands (connect, accept, reject, get, request...)

The database tracks which users are online and supplies this information to the server, which then responds to the requesting client.

3. Handshake

A handshake process occurs when a client connects to the server using TCP. This establishes a reliable session before any commands or messages are exchanged. The handshake ensures:

- Client authentication (Alias, IP Address)
- Session initialization
- Agreement on communication parameters

4. Broadcast

The server can broadcast messages to all connected clients. Broadcasts may include:

- Notifications of new users joining
- Updates when users go offline
- System announcements

Both the client and server participate in establishing connections and managing message flow between them.

Peer-to-Peer (Client–Client) Communication

In peer-to-peer communication, clients communicate directly with each other using both TCP and UDP.

1. **TCP for Text Messaging (read and write)**
   - TCP is used for text communication because text messages require reliability and ordered delivery. Since TCP establishes a connection first, it ensures that messages arrive correctly and in sequence.
2. **UDP for Media Transfer (media and files)**
   - UDP is used for media transfer (such as audio, video, or images) because media requires fast transmission with minimal delay. UDP allows rapid client-to-client data transfer without waiting for acknowledgements, making it suitable for real-time communication.

Different ports are used for TCP and UDP services, enabling separation of text and media traffic. These connections are typically non-persistent, meaning they are created when needed and closed afterward.

Transport Information and Command System

The system uses a command-based transport mechanism.

Server Commands

The server contains built-in commands that clients can invoke. Examples include:

- Registration commands
- Discovery commands
- Broadcast requests
- Status queries

When a client sends a command via TCP, the server processes it and returns a response message.

Client-Server Messaging

Communication between clients and the server mainly involves:

- Sending text commands from client to server
- Receiving structured responses from server to client

This interaction uses TCP to ensure reliability.

Client-Client Messaging

Direct communication between clients includes:

- Text messages (via TCP)
- Media transfer (via UDP)

Each type of communication uses different ports.

Roles and Responsibilities

Server Responsibilities

The server is responsible for:

- Obtaining client IP addresses and port numbers
- Storing client information in the database
- Maintaining a list of online users
- Handling client requests
- Broadcasting system messages
- Processing commands

Database Responsibilities

The database keeps track of:

- Registered users
- Currently online users
- Client identification data

When a client requests the number of online users, the server queries the database and returns the information.

Client Responsibilities

Clients are responsible for:

- Sending commands and requests to the server
- Initiating peer-to-peer connections
- Using TCP for text communication
- Using UDP for media transfer

Most requests to the server (such as discovery, broadcasting, and command usage) occur over TCP due to its reliability.

3. Design rationale and trade-off discussion

The application layer architecture has a combination of client-server and client-client communication to balance control, reliability and performance.

Design rationale

- TCP was selected for client-server communication and text messaging due to its reliability, ordered delivery and connection-oriented nature. These properties are essential for commands, authentication and text messages where accuracy and completeness are critical. The handshake mechanism in TCP ensures that connections are properly established before data exchange, improving reliability and security.

- UDP was chosen for media transfer because it provides faster transmission with lower delay time, which is essential for real-time communication. Media data such as audio, video, or images prioritizes speed and continuity over perfect reliability, making UDP an appropriate choice.

Trade-off

Reliability vs Speed

The trade-off of using TCP is increased overhead and delay caused by connection establishment and acknowledgements, which prioritise reliable data delivery. On the other hand, UDP offers speed, it sacrifices reliability and does not guarantee packet delivery. This trade-off was acceptable because media communication prioritizes speed over perfect accuracy.

Different ports are allocated for TCP and UDP communication to separate text and media traffic. Connections are non-persistent and established only when needed, saving system resources.

Overall, the design prioritizes reliability for control and messaging functions while optimizing speed for media transfer. The trade-offs between reliability, delay, centralization, and performance were carefully considered to produce a balanced and efficient communication system.

4. GenAI usage declaration

- System architecture design examples
- Deep understanding of client-server
- Deep understanding of transport layer
- Document Structuring

`